ARCHITECTURE DEEP DIVE · PRESENTATION PACK

How SmartRFP works under the hood.

The full system architecture, every step explained in plain language, how each of the two agents works internally, and the complete technology stack — built on LangGraph.

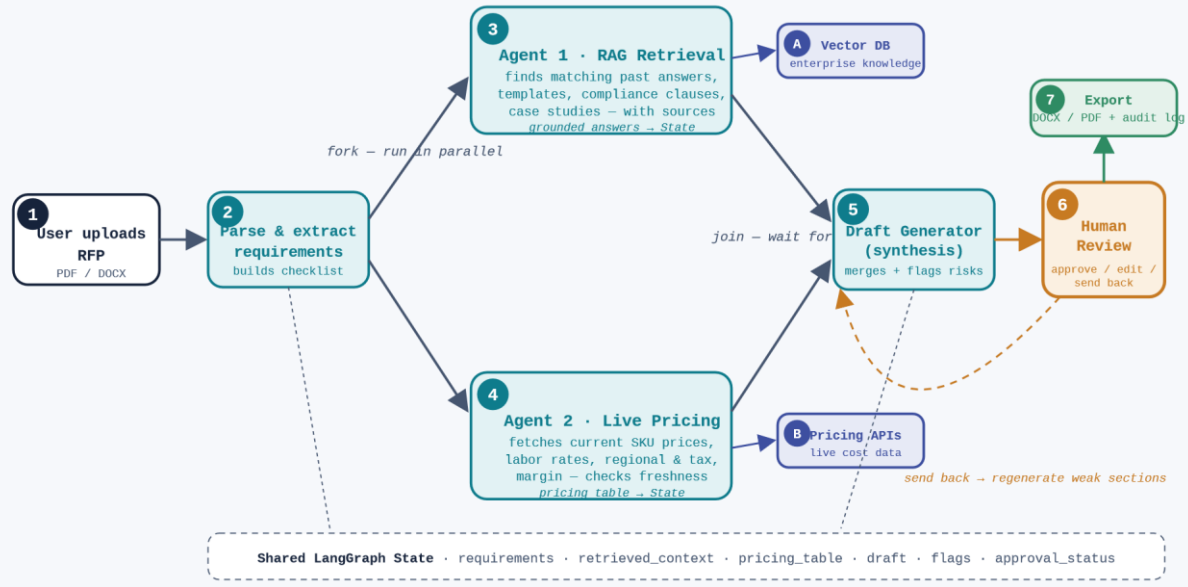
SECTION 01

The architecture at a glance

LangGraph is the "conductor." It runs a graph of nodes in a fixed order: parse the RFP, split into two agents that run **at the same time**, merge their results, pause for a human, then export. Every node reads and writes one shared **State** object that travels through the whole graph.

AI / automated step Human-in-the-loop Data & external tools

Entry / exit



SMARTRFP END-TO-END ARCHITECTURE — NUMBERS 1–7 MAP TO THE STEPS EXPLAINED ON THE NEXT PAGE

SECTION 02

Every step explained

Follow the numbers from the diagram. Each step lists what goes in and what comes out, so the data flow is easy to trace.

1

User uploads the RFP

entry point

The bid manager uploads the client's RFP file and enters the deal name, region, and deadline. This is the only manual action needed to start the whole pipeline.

IN: RFP file (PDF/DOCX) + deal details

OUT: a new workflow run with raw document

2

Parse & extract requirements

automated

The system reads the document, cleans the text, and splits it into sections. An LLM then turns those sections into a structured checklist — every question and requirement that must be answered. This checklist is what both agents work from.

IN: raw document **OUT:** list of requirements written to State

3

Agent 1 — RAG Retrieval

parallel branch

For each requirement, this agent searches the enterprise knowledge base and pulls the most relevant past content, then drafts grounded answers with the source attached. It runs at the same time as Agent 2.*(Internal pipeline in Section 03.)*

IN: requirements + Vector DB (A) **OUT:** answers + sources → State

4

Agent 2 — Live Pricing & Cost

parallel branch

This agent finds every cost-bearing item in the RFP, calls live pricing APIs, and builds a current commercial table with margin and tax applied. It checks each price's freshness. *(Internal pipeline in Section 04.)*

IN: requirements + Pricing APIs (B) **OUT:** pricing table + timestamps → State

5

Draft Generator (synthesis)

join + automated

This node waits until **both** agents finish, then merges their outputs into one structured draft: technical answers + the pricing table + a compliance section, all in a consistent house style. It also raises flags — missing info, unsupported claims (hallucination risk), and compliance items needing sign-off.

`IN:` answers + pricing table from State `OUT:` full draft + flags → State

6

Human Review gate

human-in-the-loop

The graph **pauses here** and saves its state. The reviewer opens the workspace and sees the draft with all flags highlighted. They choose one of three paths: **Approve** (continue to export), **Edit** (change inline, then approve), or **Send back** (loop only the weak sections back through the agents with a note). Nothing is finalized without this step.

`IN:` draft + flags `OUT:` decision: approve · edit · regenerate

7

Export & audit log

exit point

Once approved, the final response is exported to DOCX/PDF. An audit record stores what the AI suggested, what the human changed, and which sources and prices were used — useful evidence for the next bid and for compliance.

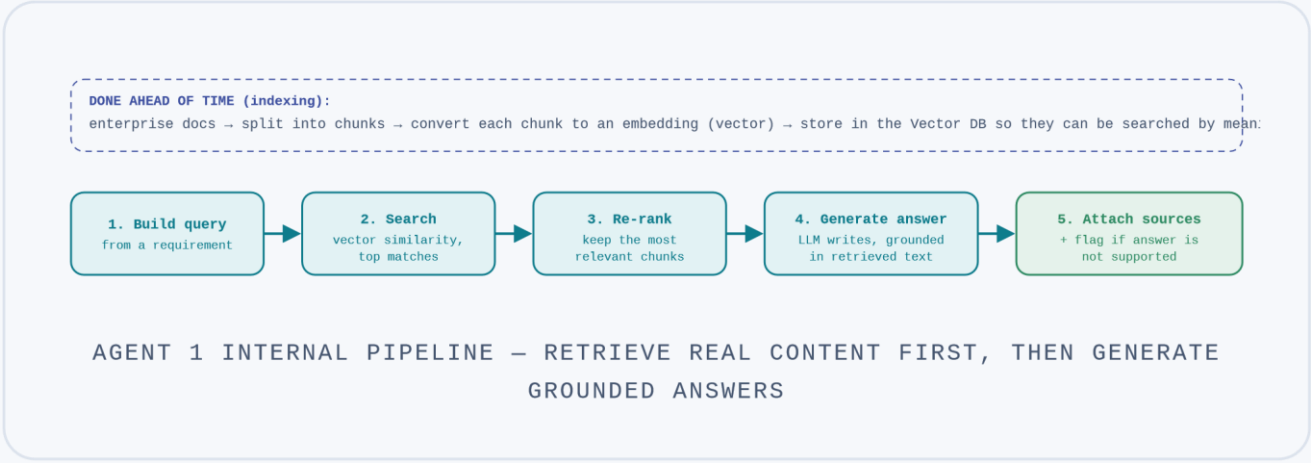
`IN:` approved draft `OUT:` final document + audit trail

> **The key idea:** steps 3 and 4 run **in parallel**, so the total time is roughly the slower agent — not the sum of both. And the graph **remembers its place** at step 6, so it can wait safely for a human and resume exactly where it stopped.

SECTION 03

How Agent 1 works — RAG Retrieval

"RAG" means Retrieval-Augmented Generation: instead of letting the AI answer from memory (which invites hallucination), we first **retrieve** real company documents, then ask the AI to write answers **grounded in those documents**, with sources attached.



AGENT 1 INTERNAL PIPELINE — RETRIEVE REAL CONTENT FIRST, THEN GENERATE GROUNDED ANSWERS

1 **Build the query.** For each requirement on the checklist (e.g. "Describe your data backup approach"), the agent forms a search query capturing the intent.

2 **Search by meaning.** The query is converted to an embedding and compared against the Vector DB. This finds documents that mean the same thing, even when the wording differs — far better than keyword search.

3 **Re-rank.** The top candidates are re-scored so only the most relevant few chunks are kept. This keeps the context tight and accurate.

4 **Generate a grounded answer.** The LLM writes the section answer using *only* the retrieved chunks as evidence, in the company's tone.

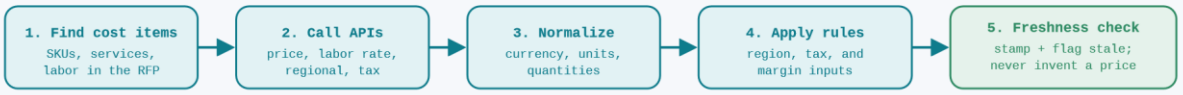
5 **Attach sources & check grounding.** Each answer carries its source document. If a claim has no supporting chunk, it is flagged as a **hallucination risk** for the human to verify.

> **What it searches:** past submitted RFP responses, proposal templates, technical solution documents, compliance documents, product/service catalog, case studies, and standard legal clauses.

SECTION 04

How Agent 2 works — Live Pricing & Cost

This agent makes sure the commercials are current. Rather than copying numbers from an old proposal, it calls live APIs at the moment of drafting and builds a fresh, traceable pricing table.



AGENT 2 INTERNAL PIPELINE — FETCH LIVE, NORMALIZE, APPLY BUSINESS RULES, VERIFY FRESHNESS

1. **Find the cost items.** The agent scans the requirements for anything that carries a cost — hardware SKUs, software licenses, managed services, and estimated labor.
2. **Call the pricing APIs.** For each item it calls the relevant API (component cost, service pricing, labor rate, regional pricing, procurement/supply, tax). Calls run together to stay fast.
3. **Normalize.** Results are converted to one currency and consistent units/quantities so the totals are comparable.
4. **Apply business rules.** Region-specific pricing, taxes, and the target margin are applied to produce sell prices.
5. **Freshness check.** Every line gets a fetch timestamp and source. If an API is down or returns an old value, the line is flagged **missing/stale** — the agent never fabricates a number.

> **Why this matters:** outdated pricing is one of the biggest risks in manual RFPs. By fetching live data and refusing to guess, Agent 2 protects the margin and the credibility of the bid.

SECTION 05

Two things that make it work: parallelism & the human gate

Running both agents in parallel

After parsing, LangGraph "forks" into two branches. Because retrieving documents (Agent 1) and calling pricing APIs (Agent 2) don't depend on each other, they run at the same time. A "join" node waits for both to finish before synthesis. Result: the draft is ready in roughly the

time of the slower agent, not both added together — about half the time of doing them one after the other.

Pausing safely for a human (checkpointing)

At the review gate, LangGraph uses an **interrupt** and saves the entire workflow State to a checkpoint store. The system can wait minutes or hours for the reviewer without losing anything. When the human decides, the graph **resumes from exactly that point** — approving moves forward to export; sending back routes only the affected sections to be regenerated with the reviewer's note as extra guidance.

SECTION 06

Technology stack

Each layer and the job it does. The labels A and B match the data sources in the architecture diagram.

LAYER	TECHNOLOGY	WHAT IT DOES IN SMARTRFP
Orchestration	`LangGraph`	Defines the agent graph, runs the two agents in parallel, manages the human-in-the-loop pause/resume and the regenerate loop, and carries the shared State.
Agent tooling	`LangChain`	Prompt templates, tool/function calling, and retriever interfaces the agents use to talk to the LLM, the Vector DB, and the pricing APIs.
Reasoning / writing	`LLM (GPT-4 / Claude)`	Extracts requirements, writes grounded answers, and synthesizes the final draft in a consistent voice.
Embeddings	`Embedding model`	Turns documents and queries into vectors so the system can search enterprise knowledge by meaning, not just keywords.
Knowledge store (A)	`Vector DB — FAISS / Pinecone / Chroma`	Stores the embedded enterprise documents and returns the most similar chunks for any query. Powers Agent 1.
Document parsing	`PyMuPDF / Unstructured / python-docx`	Reads the uploaded RFP (PDF/DOCX), extracts clean text, and splits it into sections.
Pricing data (B)	`REST APIs via httpx`	Live component cost, service pricing, labor rates, regional pricing, tax/margin. Powers Agent 2.
Backend	`FastAPI (Python)`	Runs the LangGraph workflow, exposes endpoints for upload / stacoltus / review, and serves results to the frontend.
Frontend	`Streamlit`	Upload screen, live generation status, and the human review workspace where approve/edit/send-back happens.

State & audit store	`PostgreSQL`	LangGraph checkpoints (so the workflow can pause/resume) and the audit trail of AI suggestions vs. human edits.
Caching	`Redis`	Caches recent price lookups and embeddings to cut latency and repeat API calls.
Export	`python-docx / WeasyPrint`	Produces the final approved response as DOCX and PDF.
Observability	`LangSmith`	Traces each agent run for debugging, evaluating answer quality, and monitoring in production.
Deployment	`Docker`	Ensures that the application behaves consistently across development, testing, and production environments.

> **Presenting tip:**if asked "why LangGraph and not a single prompt?", the answer is control — LangGraph gives explicit parallel branches, a reliable human-approval pause, and a regenerate loop, which a single LLM call cannot guarantee.